

**INSTITUTO MEXICANO DEL PETRÓLEO**

Coordinación Académica de Posgrado

Reporte de Participación en Proyecto

Nombre del Estudiante	Francisco Javier Cisneros Ruiz		Generación	2022	Matrícula: 92735
Fecha de elaboración	15 de junio de 2026	Tipo de Proyecto	Desarrollo de producto ✓	Tipo de financiamiento	Fondo de investigación ✓
Nivel	Doctorado ✓		Laboratorio en el que se desarrolla el proyecto:	Revisar lista anexa	
Periodo de participación	De: 4 de febrero de 2026		A: 15 de junio de 2026		
Nombre del jefe de proyecto:	Dra. Claudia Rendón Conde		Correo-e	crendonc@imp.mx	Extensión:
Número y nombre del proyecto	Procesador de Fatiga SACS — Herramienta Computacional para la Consolidación de Reportes de Análisis de Fatiga en Elementos Estructurales de Plataformas Marinas				

1. Introducción

El presente documento constituye el reporte técnico de la actividad de desarrollo de producto realizada en el Instituto Mexicano del Petróleo (IMP), consistente en el diseño, implementación, prueba y distribución de la herramienta denominada Procesador de Fatiga SACS v1.0. Dicha herramienta fue concebida para dar solución a una limitación operativa concreta del software de análisis estructural SACS (Structural Analysis Computer System, Bentley Systems): la imposibilidad de sumar automáticamente los valores de daño acumulado por fatiga reportados en archivos de salida correspondientes a distintos modelos temporales de evaluación.

En el contexto del análisis de fatiga de plataformas marinas, los ingenieros estructurales generan múltiples archivos de reporte SACS —uno por cada período de operación modelado— en los que se consignan los daños circunferenciales en ocho puntos de hotspot por elemento estructural. La consolidación de estos reportes exige sumar aritméticamente los daños correspondientes a cada elemento, identificado de manera inequívoca por la combinación JOINT–MEMBER–GRUP–tipo de soldadura–índice de incidencia. Al realizarse manualmente, este proceso resulta propenso a errores y demanda una inversión de tiempo considerable ante conjuntos de miles de elementos.

El reporte documenta el proceso de desarrollo organizado en siete etapas incrementales, las dificultades técnicas encontradas en cada fase y las soluciones adoptadas, así como las métricas de calidad obtenidas al cierre del proyecto. El periodo de participación abarca del 4 de febrero al 15 de junio de 2026.

2. Objetivo del Trabajo

El objetivo principal de la actividad consistió en desarrollar una aplicación de escritorio funcional, robusta y distribuable que permitiera a los ingenieros del IMP consolidar reportes de análisis de fatiga SACS de forma automática, confiable y reproducible. Para ello, la herramienta debía cumplir con los siguientes requisitos funcionales:

- Cargar simultáneamente N archivos .txt de salida SACS representativos de distintos períodos de evaluación de fatiga.
- Normalizar y extraer de forma automática los datos de daño registrados en notación científica Fortran (formato sin exponente explícito).
- Sumar aritméticamente el vector de daños acumulados (8 hotspots circunferenciales) por elemento estructural, preservando la identidad única JOINT–MEMBER–GRUP–weld_type–incidence_index.
- Clasificar los elementos resultantes en cuatro niveles de riesgo (CRÍTICO, ALTO, MODERADO, BAJO) conforme a umbrales definidos por el equipo técnico del IMP.
- Exportar los resultados a formato Excel profesional (.xlsx) con estilos visuales diferenciados por nivel de riesgo, incluyendo desglose por etapa y consolidado.
- Distribuir la aplicación como ejecutable independiente (.exe) para sistemas Windows, sin requerir instalación previa de Python ni de bibliotecas adicionales.

Como objetivo secundario se implementó una suite de pruebas automatizadas que garantizara la integridad de los cálculos ante cambios futuros en el código, con cobertura de todos los módulos críticos del sistema.

3. Actividades Inicialmente Planteadas

El plan de trabajo establecido contempló cuatro actividades principales articuladas en siete etapas de desarrollo incremental. La Tabla 1 resume el alcance, las fechas de inicio, la duración estimada y los entregables asociados a cada actividad.

Tabla 1. Actividades planeadas, períodos de ejecución y entregables.

Actividad	Descripción	Fecha de inicio	Duración	Entregable
Actividad 1	Exploración del formato SACS y desarrollo de módulos de limpieza, normalización y extracción de datos.	4 de febrero de 2026	1 semana	data_cleaner.py, ftg_parser.py, models.py
Actividad 2	Consolidación aritmética de daños entre archivos y clasificación de elementos por nivel de riesgo.	11 de marzo de 2026	1.5 semanas	aggregator.py, validator.py
Actividad 3	Diseño e implementación de la interfaz gráfica de usuario y módulo de exportación a Excel profesional.	20 de marzo de 2026	7 semanas	gui.py, reporte .xlsx
Actividad 4	Implementación de la suite de pruebas automatizadas, empaquetado en ejecutable Windows y documentación final.	7 de mayo de 2026	2 semanas	tests automatizados, .exe Windows, README

4. Desarrollo por Etapas

4.1 Etapa 1 — Exploración del Formato y Limpieza de Datos (4–9 de febrero de 2026)

La primera etapa tuvo por objetivo analizar en profundidad la estructura de los archivos de salida del módulo FTG de SACS y construir las funciones de preprocesamiento que permitieran convertir los datos crudos a un formato procesable por Python. El módulo resultante, `data_cleaner.py`, encapsula tres funciones fundamentales del sistema.

La función `normalize_fortran_scientific()` resuelve el desafío más inmediato: SACS exporta los valores de daño en notación científica Fortran sin exponente explícito, de la forma `.48430268-9` en lugar del formato estándar `0.48430268E-9`. Python no puede convertir directamente estas cadenas con `float()`, por lo que se implementó una expresión regular que detecta el patrón y reconstruye el valor en notación estándar antes de la conversión.

La función `is_valid_data_line()` filtra las líneas del archivo que no contienen datos estructurales: encabezados de página, separadores de sección, líneas de metadatos SACS y líneas vacías. Este filtro opera antes de cualquier intento de extracción y permite que el parser consuma únicamente información

relevante. Por su parte, `detect_file_encoding()` utiliza la biblioteca Chardet para identificar el encoding del archivo antes de abrirlo, dado que SACS puede generar salidas en UTF-8, Latin-1 o Windows-1252 según la configuración del sistema operativo donde se ejecute.

4.2 Etapa 2 — Parser con Máquina de Estados (10 de febrero de 2026)

La Etapa 2 se centró en la extracción estructurada de los datos de elementos del reporte FTG. El módulo `ftg_parser.py` implementa una máquina de estados de cuatro estados (`SEARCHING`, `READING_HEADER`, `READING_ELEMENT`, `READING_TOTAL`) que procesa el archivo línea por línea de manera secuencial y determina el contexto de cada fragmento de información.

El estado `SEARCHING` localiza el encabezado de sección `MEMBER FATIGUE DETAIL REPORT` (incluyendo variantes en formato espaciado de SACS). Una vez detectado, la máquina transita a `READING_HEADER` para esperar la línea de definición de columnas, y de ahí a `READING_ELEMENT` para extraer los identificadores `JOINT`, `MEMBER` y `GRUP`. La línea `*** TOTAL DAMAGE ***` activa el estado `READING_TOTAL`, donde se extraen y convierten los ocho valores de daño circunferencial, y se instancia el objeto `FatigueElement` correspondiente.

Un desafío relevante de esta etapa fue el campo `MEMBER`, cuyo formato es variable en los reportes SACS: puede incluir espacios internos (ej. 802L 0005), guiones en distintas posiciones (0002-501L, 401L-0002) o combinaciones alfanuméricas sin patrón fijo. La solución adoptada consistió en una estrategia de `split` inteligente que toma `JOINT` como primer token y `GRUP` como último, concatenando los tokens intermedios como `MEMBER`, lo que resulta robusto ante todas las variantes observadas.

En una iteración posterior (7 de mayo de 2026), el parser fue corregido para soportar identificadores `JOINT` alfanuméricos (ej. J576, 401L), resolviendo un fallo detectado al procesar archivos con plataformas de mayor complejidad estructural.

4.3 Etapa 3 — Consolidación Aritmética y Clasificación de Riesgo (11–20 de marzo de 2026)

La Etapa 3 implementó el núcleo de procesamiento del sistema: la suma vectorial de daños entre archivos y la clasificación por nivel de riesgo. Se definieron tres módulos principales: `models.py`, `aggregator.py` y `validator.py`.

El módulo `models.py` define la clase `FatigueElement` como `dataclass`, que encapsula los atributos de un elemento estructural (`joint`, `member`, `grup`, `weld_type`, `incidence_index`) y su vector de daños de ocho componentes como array NumPy. La clase expone propiedades calculadas: `max_damage` (máximo de los ocho hotspots), `estimated_life` (inversa del daño máximo), `critical_location` (hotspot con mayor daño) y `consistency_status` (indicador de presencia del elemento en todas las etapas procesadas).

La clase `FatigueAggregator` en `aggregator.py` recibe la lista de `ParseResult` generados por el parser para cada archivo y consolida los elementos mediante suma vectorial con NumPy: `damages_sum += element.damages` para cada etapa en la que el elemento aparece. Los elementos se identifican por su clave única (`unique_key`), garantizando que la suma se realice sobre los mismos puntos físicos entre archivos. El agregador registra también qué etapas contienen cada elemento, generando el indicador de consistencia que se muestra en la interfaz gráfica.

El módulo `validator.py` clasifica cada `FatigueElement` según umbrales definidos en sesión con el equipo técnico del IMP (actualizado el 25 de mayo de 2026): **CRÍTICO** para daño acumulado ≥ 1.0 , **ALTO** para $\geq$

0.5, MODERADO para ≥ 0.1 , y BAJO para valores menores. Estos umbrales reflejan criterios de integridad estructural acordados para las plataformas de la cartera del IMP.

4.4 Etapa 4 — Interfaz Gráfica de Usuario (20 de marzo de 2026)

La Etapa 4 produjo una aplicación de escritorio con interfaz gráfica desarrollada en Tkinter, diseñada para operar sin línea de comandos. La interfaz implementa un esquema de color oscuro con tres paneles funcionales.

El panel izquierdo (ARCHIVOS DE ENTRADA) permite al usuario agregar y eliminar archivos .txt de reportes SACS mediante los botones + Agregar y Quitar. El botón PROCESAR Y CONSOLIDAR dispara el flujo completo: lectura de archivos, normalización, extracción, agregación y clasificación, mostrando una barra de progreso y un mensaje de estado al terminar. El panel inferior izquierdo (RESUMEN) muestra estadísticas globales: total de incidencias únicas, conteo por nivel de riesgo, daño máximo global y vida mínima estimada.

El panel derecho (RESULTADOS CONSOLIDADOS) presenta una tabla ordenada por nivel de riesgo y daño descendente, con colores diferenciados por nivel: rojo para CRÍTICO, naranja para ALTO, amarillo para MODERADO y sin resalte para BAJO. Las columnas incluyen Nivel Riesgo, Joint, Member, Grup, Tipo, Inc. (índice de incidencia), Consistencia, Vida Estimada, daño máximo por etapa y Daño Total.

4.5 Etapa 5 — Exportación Excel Profesional (7 de mayo de 2026)

La funcionalidad de exportación implementa la generación de archivos .xlsx mediante la biblioteca OpenPyXL. El workbook resultante contiene una hoja CONSOLIDADO con todos los elementos ordenados por nivel de riesgo, más una hoja por cada archivo de entrada procesado con el desglose individual de daños por etapa.

Cada hoja aplica formato condicional por nivel de riesgo mediante estilos de celda: relleno rojo para CRÍTICO, naranja para ALTO, amarillo para MODERADO y verde para BAJO. Los encabezados de columna reciben formato de negrita con fondo azul oscuro y texto blanco. Las columnas numéricas de daño se formatean con notación científica de cuatro decimales para facilitar la lectura de valores del orden de 10^{-3} a 10^3 .

4.6 Etapa 6 — Suite de Pruebas Automatizadas (7 de mayo de 2026)

Se implementó una suite de pruebas con Pytest organizada por módulo y nivel de prueba. Al cierre del proyecto, la suite cuenta con 99 pruebas automatizadas que cubren los siguientes módulos:

- **test_data_cleaner.py** 31 pruebas: normalización Fortran, filtrado de líneas y detección de encoding.
- **test_ftg_parser.py** 16 pruebas: máquina de estados, extracción de identificadores y manejo de duplicados.
- **test_aggregator.py** 12 pruebas: suma vectorial, consistencia de etapas y estructura del DataFrame.
- **test_aggregator_integration.py** 13 pruebas: procesamiento de archivos reales con más de 2,300 incidencias.
- **test_validator.py** 21 pruebas: clasificación por umbrales, conteos de riesgo y resumen estadístico.

Las pruebas de integración operan sobre los tres archivos de datos reales de la plataforma marina del caso de estudio, con un total combinado de más de 500,000 líneas de texto. El umbral de aceptación para cada commit es el 100 % de pruebas pasando.

4.7 Etapa 7 — Empaquetado y Distribución Windows (7–18 de mayo de 2026)

La etapa final de desarrollo técnico consistió en generar un ejecutable independiente para sistemas Windows mediante PyInstaller 6+. Se elaboró un archivo de especificación procesador_fatiga_sacs.spec que define el modo onefile (un único .exe), los hooks de Tkinter, NumPy, Pandas y OpenPyXL, y el icono de la aplicación.

El script de construcción build_windows.bat automatiza el proceso en el entorno Conda del desarrollador: activa el entorno, instala las dependencias, ejecuta PyInstaller y valida la existencia del ejecutable resultante en el directorio dist/. El ejecutable final no requiere instalación de Python ni de ninguna biblioteca en el equipo del usuario final, lo que facilita su despliegue en los entornos de trabajo del IMP.

4.8 Mejoras Post-release por Retroalimentación del Equipo IMP (junio de 2026)

Tras la entrega de la versión v1.0 (18 de mayo de 2026), el equipo técnico del IMP realizó sesiones de revisión que derivaron en un conjunto de mejoras implementadas entre el 9 y el 15 de junio de 2026.

La mejora más significativa fue la distinción de pares BRACE/CORD en el parser. SACS reporta dos bloques de daño por cada unión tubular: uno para la soldadura tipo BRACE (diagonal o arriostrado) y otro para la soldadura tipo CORD (cordón o tubo principal). En la versión inicial, ambos bloques se fusionaban bajo la misma clave, perdiendo información crítica. La solución implementada asigna weld_type (BRACE o CORD) e incidence_index según el orden de aparición en el reporte, generando claves únicas diferenciadas para cada tipo de soldadura. Este cambio elevó el total de incidencias consolidadas de 1,949 a 2,304.

Adicionalmente se incorporaron: desglose del daño máximo por etapa individual en el panel de resultados, indicador de consistencia por elemento (OK/PARCIAL) y la columna Vida Estimada (1/daño_max) como métrica de vida remanente. En el módulo de robustez, se corrigió el reinicio del contador de ocurrencias al inicio de cada sección del reporte y se añadió logging explícito para elementos duplicados con daños distintos.

5. Resultados Obtenidos

La herramienta fue validada con datos reales de operación correspondientes a tres períodos de evaluación de fatiga de una plataforma marina de la cartera del IMP. Los archivos de entrada totalizan 506,239 líneas de texto y fueron procesados en su totalidad sin errores de parsing. La Tabla 2 presenta la distribución de los 2,304 elementos consolidados por nivel de riesgo.

Tabla 2. Distribución de incidencias consolidadas por nivel de riesgo.

Nivel de Riesgo	Criterio (daño acumulado)	Incidencias	% del total
CRÍTICO	≥ 1.0	149	6.5 %
ALTO	≥ 0.5 y < 1.0	46	2.0 %
MODERADO	≥ 0.1 y < 0.5	90	3.9 %
BAJO	< 0.1	2,019	87.6 %
TOTAL	—	2,304	100 %

El elemento con mayor daño acumulado corresponde al nodo 0187, miembro 0374–0187, grupo V3, soldadura tipo CORD, con un daño consolidado de 3,904.37 en la posición circunferencial LEFT (hotspot

crítico). El período de operación 2 (Etapa 2) contribuye con el 83.8 % del daño total de ese elemento; la Etapa 1 no registra ese miembro, lo que se refleja en una consistencia PARCIAL.

En cuanto al desempeño del procesamiento, los tres archivos fueron parseados y consolidados en menos de 8 segundos en una máquina de escritorio estándar (procesador Intel Core i5, 16 GB RAM), lo que resulta adecuado para el flujo de trabajo de los ingenieros del IMP.

Tabla 3. Pila tecnológica del sistema.

Componente	Tecnología	Rol en el sistema
Lenguaje base	Python 3.11+	Lógica de negocio completa
Interfaz gráfica	Tkinter	GUI standalone multiplataforma
Análisis numérico	NumPy	Suma vectorial de daños (8 hotspots)
Manipulación tabular	Pandas	DataFrame para consolidación y exportación
Exportación Excel	OpenPyXL	Formato profesional .xlsx con estilos
Detección de encoding	Chardet	Lectura robusta de archivos SACS
Testing	Pytest	Suite automatizada de 99 pruebas
Empaquetado	PyInstaller 6+	Ejecutable .exe sin dependencias Python
Gestión de entorno	Conda / Mamba	environment.yml reproducible

6. Dificultades Técnicas Encontradas

A lo largo del desarrollo se identificaron cuatro dificultades técnicas principales, cuya resolución resultó determinante para la confiabilidad y precisión de los resultados.

6.1 Notación Científica Fortran sin exponente explícito

SACS exporta los valores numéricos de daño en formato Fortran puro, donde el exponente se escribe directamente adyacente a la mantisa sin la letra E ni el cero inicial: .48430268-9 en lugar de 0.48430268E-09. Ninguna función nativa de Python (float(), numpy.float64) reconoce este formato. La solución implementada detecta mediante expresión regular la presencia de un signo (+ o -) no precedido de E, reconstruye la cadena en formato estándar y la convierte. Se validó con más de 50 casos de prueba unitaria, incluyendo valores positivos, negativos, con y sin parte entera.

6.2 Campo MEMBER de formato variable

El identificador de miembro estructural en los reportes SACS no sigue un ancho de campo fijo: puede contener espacios internos (802L 0005), guiones en distintas posiciones (0002-501L, 401L-0002) o ser una cadena alfanumérica continua. Un split simple por espacios produce tokens incorrectos cuando el campo contiene espacios. La estrategia adoptada interpreta siempre el primer token de la línea como JOINT y el último como GRUP, concatenando los tokens intermedios como MEMBER, lo que preserva la integridad del campo independientemente del formato encontrado.

6.3 Identificadores JOINT alfanuméricos

Los archivos iniciales de prueba contenían exclusivamente identificadores JOINT numéricos de cuatro dígitos (0003, 0187, etc.). Al procesar archivos de plataformas distintas se detectaron JOINTs alfanuméricos (J576, 401L, J101), que el parser rechazaba al intentar validarlos como enteros. La corrección (7 de mayo de 2026) amplió la expresión regular de identificación para aceptar cualquier secuencia alfanumérica, incrementando el total de elementos reconocidos de 1,722 a 2,302 en los archivos de prueba.

6.4 Pares BRACE/CORD en el reporte de daño total

SACS genera dos bloques *** TOTAL DAMAGE *** consecutivos para la misma combinación JOINT–MEMBER–GRUP cuando la unión es tubular: el primero corresponde a la soldadura del arriostre (BRACE) y el segundo al cordón principal (CORD). La versión inicial del parser los trataba como duplicados y los fusionaba conservando el máximo, lo que suprimía la mitad de la información disponible. La solución implementada mantiene un contador de ocurrencias por combinación base: las ocurrencias impares corresponden a BRACE y las pares a CORD, y el índice de incidencia (Inc.) numera los pares sucesivos. Este mecanismo elevó el total de incidencias únicas en un 18.2 %.

7. Conclusiones

La actividad de desarrollo alcanzó la totalidad de los objetivos propuestos. Se entregó una herramienta funcional, probada y distribuida como ejecutable Windows independiente, que resuelve de forma automatizada la consolidación de reportes de análisis de fatiga SACS. La suite de 99 pruebas automatizadas —todas pasando al cierre del proyecto— provee una base de confianza sólida para el mantenimiento futuro del código.

Desde el punto de vista técnico, el proyecto representó un ejercicio integral de ingeniería de software aplicada a procesamiento de datos de ingeniería estructural: comprensión profunda del formato de salida de un software especializado, diseño de una máquina de estados para parsing robusto, procesamiento vectorial con NumPy, diseño de interfaz gráfica y empaquetado para distribución en entornos de producción.

Las mejoras incorporadas tras la retroalimentación del equipo IMP —especialmente la distinción BRACE/CORD y el índice de incidencia— evidencian que el diseño modular adoptado facilita la evolución del sistema sin comprometer la integridad de los resultados ya validados. El incremento del 18.2 % en el conteo de incidencias únicas, lejos de representar un error previo, refleja una comprensión más precisa de la semántica de los reportes SACS y una mejor fidelidad al modelo físico de la estructura.

Como trabajo futuro se identifica la posibilidad de extender la herramienta para soportar múltiples plataformas en una misma sesión, incorporar visualización gráfica de la distribución de daño en la estructura y generar reportes PDF directamente desde la aplicación.



INSTITUTO MEXICANO DEL PETRÓLEO
Coordinación Académica de Posgrado
Reporte de Participación en Proyecto

	V.º B.º del asesor	
	<i>Vo. Bo. Supervisor Técnico</i> Dr. Izael Nava Viveros	

Director de Tesis	Alumno	Jefe de Proyecto
		
Dr. Iván Félix González	Francisco Javier Cisneros Ruiz	Dra. Claudia Rendón Conde